Practical:12 **Date:30-10-2026**

Developing Multiple Linear Regression from Scratch

Procedure:

Step 1: Import Libraries

Import NumPy, Pandas, Matplotlib and Scikit-learn libraries required for data analysis, calculations and visualization.

Step 2: Load the Dataset

Load the house price dataset using Pandas and display the column names.

Step 3: Extract Features and Target Variable

Extract the first two columns as input features (house size and number of bedrooms) and the third column as the target variable (house price).

Step 4: Display Training Examples

Calculate the total number of training examples and display the first five records.

Step 5: Visualize the Training Data

Create a scatter plot to study the relationship between house size and house price.

Step 6: Perform Feature Normalization

Calculate the mean and standard deviation of each feature and normalize the input variables to bring them to a common scale.

Step 7: Add Intercept Term

Add a column of ones to the feature matrix to include the intercept term in the linear regression model.

Step 8: Define the Cost Function

Define the cost function to calculate the Mean Squared Error-based cost for the linear regression model.

Step 9: Define the Gradient Descent Function

Define the gradient descent algorithm to update the model parameters (theta) iteratively and minimize the cost function.

Step 10: Initialize Parameters

Initialize theta with zeros and specify the learning rate (alpha = 0.15) and number of iterations (400).

Step 11: Apply Gradient Descent

Execute the gradient descent algorithm to obtain the optimized parameter values and cost history.

Step 12: Plot Convergence of Gradient Descent

Plot the cost function against the number of iterations to observe how the cost decreases during optimization.

Step 13: Study the Effect of Learning Rate

Repeat gradient descent using different learning rates (0.005, 0.01, 0.02, 0.03, 0.15 and 1.32) and compare their convergence curves.

Step 14: Calculate Actual and Predicted Values

Use the optimized parameters to calculate predicted house prices and compare them with actual values.

Step 15: Calculate R^2 Score

Calculate the R^2 score to evaluate the goodness of fit of the gradient descent model.

Step 16: Apply Linear Regression Using Scikit-learn

Build and train a Linear Regression model using Scikit-learn's LinearRegression() function.

Step 17: Display Model Coefficients and Intercept

Obtain and display the estimated coefficients and intercept of the fitted regression model.

Step 18: Predict House Prices

Use the trained Scikit-learn model to predict house prices and compare actual and predicted values.

Step 19: Evaluate the Model

Calculate the R^2 score using Scikit-learn to assess the model's performance.

Solution:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn import linear_model
# Load Dataset
df = pd.read_csv(r"C:\Users\Kavya Shettigar\Downloads\data.csv.txt")
print(df.columns.tolist())
# Target and Feature Metric
X = df.values[:, 0:2]
y = df.values[:, 2]
m = len(y)
print('Total no of training examples (m) = %s \n' % (m))
# Show first 5 records
for i in range(5):
    print('x = ', X[i], 'y = ', y[i])
```

```
# Scatter Plot
X1 = df.iloc[:, 0]
Y1 = df.iloc[:, 2]
plt.scatter(X1, Y1, color='red', marker='+')
plt.grid()
plt.rcParams["figure.figsize"] = (10, 6)
plt.xlabel('Size (in square feet)')
plt.ylabel('Price')
plt.title('Scatter plot of training data')
plt.show()
# Feature Normalization
def feature_normalize(X):
    mu = np.mean(X, axis=0)
    sigma = np.std(X, axis=0, ddof=1)
    X_norm = (X - mu) / sigma
    return X_norm, mu, sigma
X, mu, sigma = feature_normalize(X)
print('mu= ', mu)
print('sigma= ', sigma)
print('X_norm= ', X[:5])
```

```
# Add column of ones
X = np.hstack((np.ones((m, 1)), X))
print(X[:5])
# Compute Cost Function
def compute_cost(X, y, theta):
    predictions = X.dot(theta)
    errors = np.subtract(predictions, y)
    J = 1 / (2 * m) * errors.T.dot(errors)
    return J
# Gradient Descent Function
def gradient_descent(X, y, theta, alpha, iterations):

    cost_history = np.zeros(iterations)
    for i in range(iterations):
        predictions = X.dot(theta)
        errors = np.subtract(predictions, y)
        sum_delta = (alpha / m) * X.transpose().dot(errors)
        theta = theta - sum_delta
        cost_history[i] = compute_cost(X, y, theta)
    return theta, cost_history
```

```
# Initialize Parameters
theta = np.zeros(3)
iterations = 400
alpha = 0.15
theta, cost_history = gradient_descent(X, y, theta, alpha, iterations)
print('Final value of theta =', theta)
print('First 5 values from cost_history =', cost_history[:5])
print('Last 5 values from cost_history =', cost_history[-5:])
# Convergence of Gradient Descent
plt.plot(range(1, iterations + 1), cost_history, color='blue')
plt.grid()
plt.xlabel('Number of iterations')
plt.ylabel('Cost (J)')
plt.title('Convergence of gradient descent')
plt.show()
```

```
# Effect of Learning Rate on Convergence
iterations = 400
theta = np.zeros(3)
alpha = 0.005
theta_1, cost_history_1 = gradient_descent(X, y, theta, alpha, iterations)
alpha = 0.01
theta_2, cost_history_2 = gradient_descent(X, y, theta, alpha, iterations)
alpha = 0.02
theta_3, cost_history_3 = gradient_descent(X, y, theta, alpha, iterations)
alpha = 0.03
theta_4, cost_history_4 = gradient_descent(X, y, theta, alpha, iterations)
alpha = 0.15
theta_5, cost_history_5 = gradient_descent(X, y, theta, alpha, iterations)
alpha = 1.32
theta_6, cost_history_6 = gradient_descent(X, y, theta, alpha, iterations)
plt.plot(range(1, iterations + 1), cost_history_1, color='purple', label='alpha = 0.005')
plt.plot(range(1, iterations + 1), cost_history_2, color='red', label='alpha = 0.01')
plt.plot(range(1, iterations + 1), cost_history_3, color='green', label='alpha = 0.02')
plt.plot(range(1, iterations + 1), cost_history_4, color='yellow', label='alpha = 0.03')
plt.plot(range(1, iterations + 1), cost_history_5, color='blue', label='alpha = 0.15')
plt.plot(range(1, iterations + 1), cost_history_6, color='brown', label='alpha = 1.32')
plt.grid()
```

```
plt.xlabel('Number of iterations')
plt.ylabel('Cost (J)')
plt.title('Effect of Learning Rate on Convergence of Gradient Descent')
plt.legend()
plt.show()
# Actual vs Predicted Values using Gradient Descent
predict = np.round(X.dot(theta_5), 1)
Act_Predict = np.stack((y, predict), axis=1)
print('Actual vs Predicted (First 5):')
print(Act_Predict[:5])
# R2 Score
def r2score(predict, y):
    rss = np.sum((predict - y) ** 2)
    tss = np.sum((y - y.mean()) ** 2)
    r2 = 1 - (rss / tss)
    return r2
print('Gradient Descent R2 Score:',
      np.round(r2score(predict, y), 2))
```

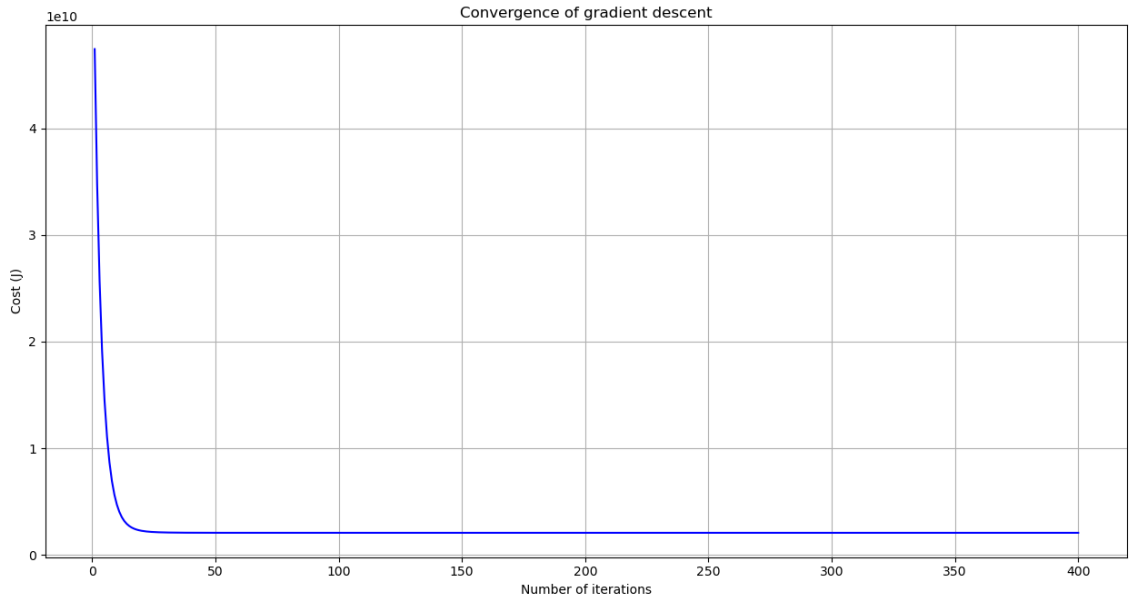
```
# Linear Regression using Scikit-learn
model_ols = linear_model.LinearRegression()
model_ols.fit(X, y)
coef = model_ols.coef_
intercept = model_ols.intercept_
print('coef= ', coef)
print('intercept= ', intercept)
# Actual vs Predicted using Scikit-learn
y_pred = model_ols.predict(X)
Act_Predict1 = np.stack((y, y_pred), axis=1)
print('Actual vs Predicted using Scikit-learn (First 5):')
print(Act_Predict1[:5,])
# Calculate R2 Score
print('Scikit-learn Score: ',
      np.round(model_ols.score(X, y), 2))
```

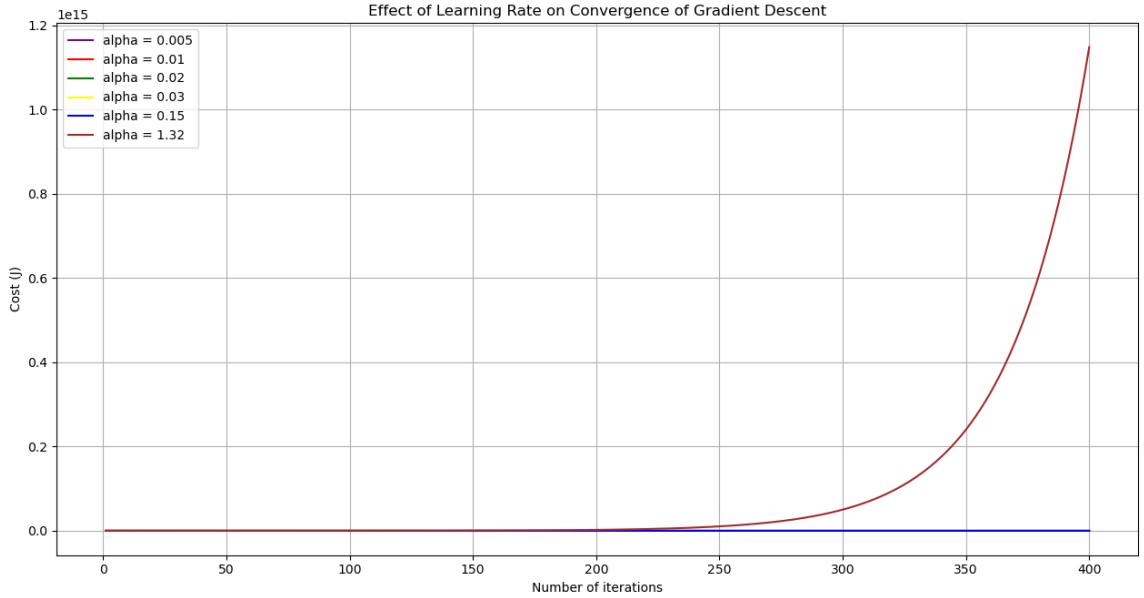
```
['2104', '3', '399900']
Total no of training examples (m) = 46

x =  [1600    3] y =  329900
x =  [2400    3] y =  369000
x =  [1416    2] y =  232000
x =  [3000    4] y =  539900
x =  [1985    4] y =  299900
mu=  [1998.43478261    3.17391304]
sigma=  [8.03333019e+02 7.68963164e-01]
X_norm=  [[-0.4959771  -0.22616564]
 [ 0.49987391 -0.22616564]
 [-0.72502283 -1.52661805]
 [ 1.24676217  1.07428677]
 [-0.0167238   1.07428677]]
[[ 1.         -0.4959771  -0.22616564]
 [ 1.          0.49987391 -0.22616564]
 [ 1.         -0.72502283 -1.52661805]
 [ 1.          1.24676217  1.07428677]
 [ 1.         -0.0167238   1.07428677]]
Final value of theta = [339119.45652174 111467.17877034  -6295.02687911]
First 5 values from cost_history = [4.74239977e+10 3.47147936e+10 2.56380449e+10 1.91359101e+10
 1.44651847e+10]
Last 5 values from cost_history = [2.06650278e+09 2.06650278e+09 2.06650278e+09 2.06650278e+09
 2.06650278e+09]
Actual vs Predicted (First 5):
[[329900.  285258. ]
 [369000.  396262.7]
 [232000.  267913.3]
 [539900.  471329.9]
 [299900.  330492.6]]
Gradient Descent R2 Score: 0.73
```

```
coef=  [     0.         111467.1787705   -6295.02687927]
intercept=  339119.45652173914
Actual vs Predicted using Scikit-learn (First 5):
[[329900.         285258.00703053]
 [369000.         396262.7097631 ]
 [232000.         267913.30827799]
 [539900.         471329.85393658]
 [299900.         330492.63734463]]
Scikit-learn Score:  0.73
```




Convergence of gradient descent
1e10
Cost (J)
Number of iterations


Effect of Learning Rate on Convergence of Gradient Descent
1e15
alpha = 0.005
alpha = 0.01
alpha = 0.02
alpha = 0.03
alpha = 0.15
alpha = 1.32
Cost (J)
Number of iterations